

本小节内容

2019 年 41 题代码实战

2019 年 41 题代码实战

上一小节我们针对真题已经进行了题目解读与设计，代码流程是我们先用尾插法新建一条链表，然后将链表拆分为两条，分别为 L 和 L2，然后 L2 进行逆置，再把 L 和 L2 进行合并，下面我们直接上代码

```
#include <stdio.h>
#include <stdlib.h>

typedef int ElemType;
typedef struct LNode {
    ElemType data;
    struct LNode* next; //指向下一个结点
}LNode, * LinkList;

//尾插法新建链表
LinkList list_tail_insert(LinkList& L)
{
    int x;
    L = (LinkList)malloc(sizeof(LNode)); //带头节点的链表
    LNode* s, * r = L; //LinkList s, r=L; 也可以，r 代表链表表尾结点，指向链表尾部
    //3 4 5 6 7 9999
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode*)malloc(sizeof(LNode));
        s->data = x;
        r->next = s; //让尾部结点指向新结点
        r = s; //r 指向新的表尾结点
        scanf("%d", &x);
    }
    r->next = NULL; //尾结点的 next 指针赋值为 NULL
    return L;
}

//打印链表
void PrintList(LinkList L)
{
    L = L->next;
    while (L != NULL) //NULL 是为了代表一张空的藏宝图
```



```
{  
    printf("%3d", L->data); //打印当前结点数据  
    L = L->next; //指向下一个结点  
}  
printf("\n");  
}
```

```
void find_middle(LinkList L, LinkList& L2)  
{  
    L2 = (LinkList)malloc(sizeof(LNode)); //带头节点的链表  
    LinkList pcur, ppre;  
    pcur = ppre = L->next;  
    while (pcur != NULL)  
    {  
        pcur = pcur->next;  
        if (NULL == pcur)  
        {  
            break;  
        }  
        pcur = pcur->next; //大哥走两步  
        if (NULL == pcur)  
        {  
            break;  
        }  
        ppre = ppre->next; //小弟走一步  
    }  
    L2->next = ppre->next; //让 L2 称为链表后面的一半  
    ppre->next = NULL; //让 L 成为前一半  
}
```

```
void reverse(LinkList L2)  
{  
    LinkList r, s, t;  
    r = s = t = L2->next;  
    if (NULL == r) //链表为空  
    {  
        return;  
    }  
    s = t = t->next;  
    if (NULL == t) //链表只有 1 个结点  
    {  
        return;  
    }  
}
```

```
t = t->next;
while (t != NULL)
{
    s->next = r;
    r = s;
    s = t;
    t = t->next;
}
s->next = r;
L2->next->next = NULL; //链表的第一个结点的 next 要为 NULL
L2->next = s; //s 是头部
}
```

```
void merge(LinkList L, LinkList L2)
{
    LinkList pcur, p, q;
    pcur = p = L->next;
    q = L2->next;
    p = p->next;
    while (q!=NULL&& p!=NULL)
    {
        pcur->next = q; //链表 L 给过来一个结点
        q = q->next; //它就往后走一个
        pcur = pcur->next; //新链表往后走一个
        pcur->next = p; //链表 L2 给过来一个结点
        p = p->next; //它就往后走一个
        pcur = pcur->next; //新链表往后走一个
    }
    if (q != NULL)
    {
        pcur->next = q;
    }
    if (p != NULL)
    {
        pcur->next = p;
    }
}
```

```
int main()
{
    LinkList L; //链表头，是结构体指针类型
    LinkList search; //用来存储拿到的某一个节点
    list_tail_insert(L); //输入数据可以为 3 4 5 6 7 9999
    PrintList(L); //链表打印
}
```

```
//寻找中间结点，并返回第二条链表
LinkedList L2=NULL;
find_middle(L, L2);
printf("-----\n");
PrintList(L);
PrintList(L2);
printf("-----\n");
reverse(L2);
PrintList(L2);
printf("-----\n");
merge(L, L2);
free(L2);
PrintList(L);
return 0;
}
```